
MCP · Skills로 Agent 능력 개선하기

실행 능력을 연결하고, 업무 절차를 평가하고, 검증된 skill을 조직에 배포합니다.

Agent의 능력은 어디에서 개선될까요?

모델 교체만이 해법은 아닙니다. 정보 접근.절차.정확한 실행.반복 품질을 나눕니다.

ACCESS

실행 범위

사내 문서·티켓·API에
접근할 수 있는가?

MCP로 능력 연결

PROCEDURE

업무 수행법

언제 무엇을 어떤
순서로 해야 하는가?

Skills로 절차 추적

EXECUTION

정확한 실행

계산·검사·상태
변경을
정확히 처리하는가?

Tool / Script

IMPROVEMENT

반복 품질

수정 후에도
더 잘 수행하는가?

Eval → 검토 → 공유

MCP와 Skills는 모델을 재학습하지 않습니다. 개선 대상은 Agent 시스템의 수행 능력입니다.

한 장으로 보는 MCP와 Skills의 조합

Skill은 실행 프로그램이 아니라 절차 자원입니다. Agent가 이를 읽고 Tool을 선택합니다.



MCP가 제공한 실행 능력을, Skill이 안내하는 절차에 따라 Agent가 사용합니다.

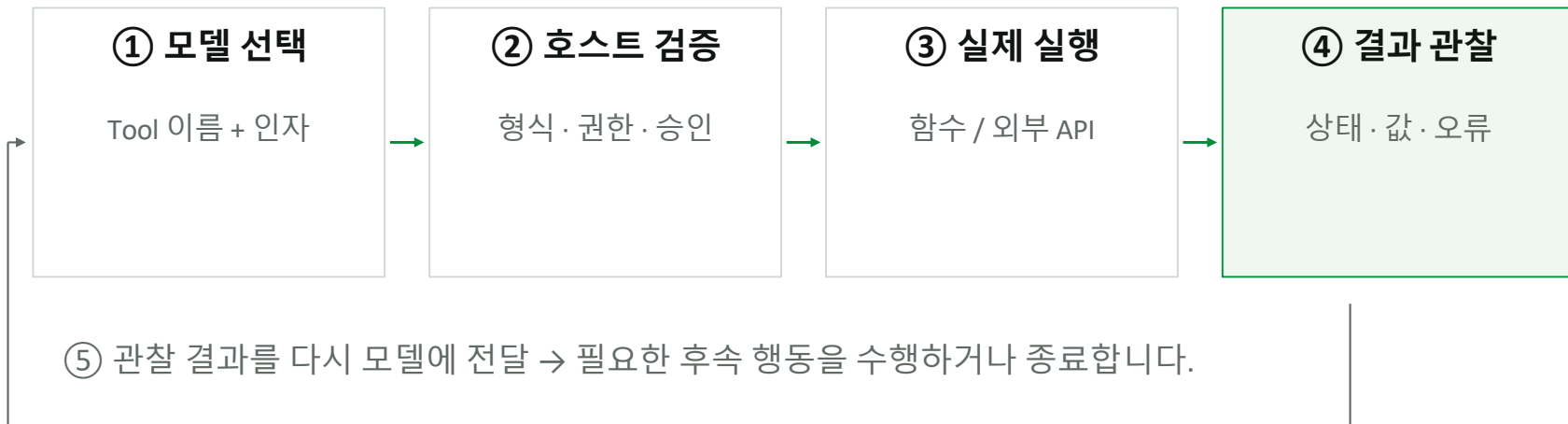
Chapter 1.

답변을 넘어 행동으로

도구를 붙였는데도 업무를 끝내지 못하는 이유는 무엇일까요?

모델은 요청하고, 호스트는 실행합니다

Function Calling은 자연어 판단과 외부 시스템의 실행 사이에 구조화된 계약을 만듭니다.



“처리했습니다”라는 문장이 아니라, 실행 결과와 외부 상태를 확인합니다.

같은 Agent를 네 단계로 확장해 봅니다

추가되는 능력과 여전히 남는 한계를 함께 봅니다. 아래는 설계 비교이며 실측 결과가 아닙니다.

단계	추가하는 것	새로 가능한 일	남는 확인 과제
A · 기본 Agent	모델 + 기본 지시	제공된 정보로 설명	외부 기록 접근 불가
B · + MCP Tools	검색·티켓 기능 연결	기록 조회·상태 확인	올바른 절차를 따르는가?
C · + Skills	업무별 절차·자료	근거·정책·검증을 조합	선택·수행 누락은 없는가?
D · + Evaluation	고정 평가셋·회귀 검사	실패를 찾아 최소 수정	개선과 퇴행을 구분

MCP 연결은 접근 범위를, Skill은 수행 방식을 바꿉니다. 성공 여부는 따로 평가합니다.

Tools · MCP · Skills는 다른 층입니다

실행 단위, 표준 연결 경계, 재사용 절차를 나누면 수정할 위치가 명확해집니다.

01

Tool

무엇을 실행할 것인가?

search_policy(query)
validate_timeline(events)

02

MCP

어떤 표준으로 연결할까?

문서·티켓 서비스
독립 배포 · 여러 Host

03

Skill

언제 어떤 절차를 따를까?

근거 → 정책 → 검증
→ 보고 / 승인 후 반영

MCP나 Skill이냐가 아닙니다. MCP Tool을 Skill과 함께 사용하는 구조가 가능합니다.

Chapter 2.

Agent가 잘 쓰는 Tool

Tool 이 단순히 동작하는 것과 Agent가 잘 사용하는 것은 무엇이 다를까요?

Deep Agents는 실행을 연결하는 Harness입니다

모델·파일·도구·Skill을 연결하되, 업무 규칙과 실행 통제의 위치를 분리합니다.



공통 통제: 인증 · 권한 · 승인 · 예산 · 감사 / 실행 경로에서 적용

모델은 판단을 제안합니다. 실제 접근 범위와 변경 가능 범위는 실행 환경이 결정합니다.

좋은 Tool은 선택 가능한 계약입니다

이름·설명·인자를 보고 언제 호출해야 하는지 이해할 수 있어야 합니다.

불분명한 계약

```
def find_policy(q):  
    ...
```

개선한 계약 · 구현부 생략

```
def search_policy(  
    query: str,  
    top_k: int = 5,  
    include_archived: bool = False,  
) -> list[dict]:  
    """Search authoritative policy evidence.  
    Use when a policy decision needs sources.  
    Return source IDs and short excerpts.  
    """  
    ...
```

무슨 행동인지는 Tool description에, 여러 행동의 순서는 skill에 둡니다.

Tool 결과는 다음 판단의 재료입니다

핵심 값과 상태·출처를 작게 반환하고, 자료 없음과 실행 실패를 구분합니다.

검색 응답 · 교육용 계약

```
{
  "status": "ok",
  "index_version": "snap-02",
  "evidence": [{
    "source_id": "E01",
    "observed_at": "00:01 UTC",
    "summary": "Error rate increased"
  }]
}
```

돌려줘야 할 정보

결과: 무엇을 관측했는가?
출처: 어디에서 가져왔는가?
조건: 어느 시점·버전인가?
상태: 성공·빈 결과·오류인가?

불필요한 원문 덤프는 줄입니다.

구조화된 데이터가 있다는 사실과 모델이 그 내용을 실제로 받았다는 사실은 다릅니다.

함수의 정확성과 Agent의 사용을 나눕니다

먼저 결정적 계약을 검증한 뒤, 모델의 선택과 인자 생성을 확인합니다.

1 · WITHOUT MODEL

Tool / Script

정상·경계·실패 입력
타입·시간대·근거 ID

같은 입력 → 같은 기대값

2 · WITH MODEL

선택과 인자

필요한 Tool을 골랐는가?
정확한 입력을 만들었는가?

불필요한 호출도 평가

3 · END-TO-END

업무 완료

관측 결과로 요청을 해결했나?
미확인 사실을 구분했나?

권한·근거·실제 상태 확인

단위 테스트 통과는 Tool 구현의 증거입니다. 올바른 Tool 선택과 업무 성공은 별도입니다.

Lab A. Tool 계약을 직접 구현합니다

예제 확인 → 핵심 코드 작성 → 테스트 → 실패 원인 설명 순서로 진행합니다.

40

MINUTES

10분 예제 확인 · 입력과 출력 계약 작성

15분 검색 / 시간순 검증 로직 구현

10분 정상·경계·오류 테스트 작성

5분 호출 기록과 실패 원인 리뷰

제출물

Tool 2개 · 단위 테스트
선택 요청 5개

완료 기준

자료 없음과 오류를 구분
경계 입력의 기대값을 설명

완성 함수 호출만으로 끝내지 않습니다. 핵심 로직과 검증 조건을 직접 작성합니다.

Chapter 3.

Skills로 업무 수행법을 축적합니다

긴 프롬프트를 파일로 옮기는 것과, 재사용 가능한 skill은 무엇이 다를까요?

skill이 개선하는 것은 두 가지입니다

더 잘 수행하게 만드는 기법과, 우리 업무 방식에 맞추는 절차를 구분합니다.

① 수행 능력을 보완합니다

혼자서는 불안정하던 작업에
검증된 기법과 자원을 제공합니다.

예: 근거 대조 · 시간순 검증
확인: skill 없이도 같은 품질인가?

② 업무 방식을 일관되게 합니다

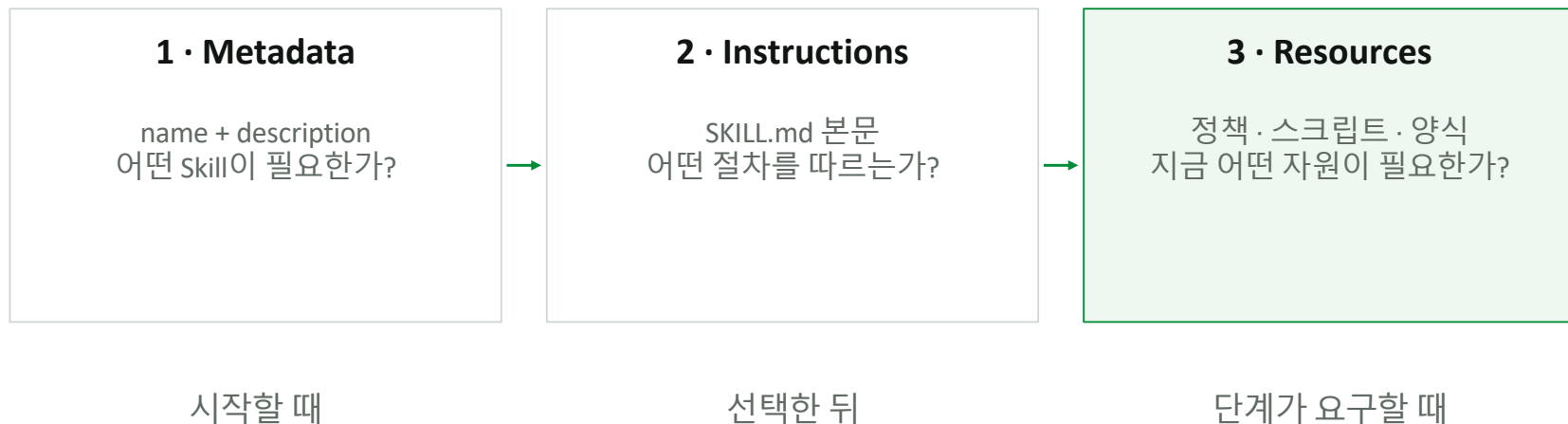
가능한 여러 행동을 우리 업무의
판단 기준과 순서로 묶습니다.

예: 정책 판정 · 승인 후 티켓 반영
확인: 실제 업무 절차를 지켰는가?

좋은 skill은 오래 남는 긴 프롬프트가 아니라, 계속 검증할 수 있는 업무 자산입니다.

Progressive disclosure · 필요한 만큼만 읽기

선택 정보, 수행 지침, 보조 자원을 분리해 필요한 시점에 컨텍스트를 확장합니다.



모든 본문을 처음부터 넣지 않습니다. 그러나 Skill의 metadata도 컨텍스트 비용이 듭니다.

skill은 절차와 자원이 함께 있는 묶음입니다

최소 단위는 SKILL.md입니다. 필요한 정책.스크립트.양식은 목적에 맞게 분리합니다.

권장 폴더 구성

```
skills/  
  incident-analysis/  
    SKILL.md  
  references/  
    severity-policy.md  
  scripts/  
    validate_timeline.py  
  assets/  
    report-template.md
```

위치

답을 내용

SKILL.md

선택 조건 · 절차 · 분기 · 완료

references/

상세 정책 · 도메인 참고자료

scripts/

계산 · 파싱 · 결정적 검사

assets/

보고서 양식 · 정적 자원

파일이 있다는 것만으로 활용되지는 않습니다. SKILL.md에서 언제 사용할지 연결합니다.

Description은 선택을 위한 계약입니다

무엇을 하는지만 쓰지 말고, 어떤 요청에서 선택해야 하는지도 적습니다.

모호한 설명

Helps with research.

어떤 조사인가?
언제 선택하는가?
무엇을 결과로 내는가?

대상 + 요청 조건 + 결과

운영 장애·지연 급증·사후 분석 요청에서
관련 근거로 영향과 시간순 경과를 재구성하고,
정책에 따라 심각도와 미확인 원인을 정리한다.

개념 설명이나 신규 정책 작성은 별도 업무다.

관련 단어가 등장했다고 같은 업무는 아닙니다. 선택해야 할 때와 아닐 때를 함께 평가합니다.

SKILL.md · 선택 조건과 수행 조건을 씁니다

“잘 분석한다”를 실제로 관찰하고 확인할 수 있는 단계로 바꿉니다.

선택 조건 · YAML frontmatter

```
---
name: incident-analysis
description: >
  Analyze operational incidents
  using source evidence and
  severity policy. Use for
  outage reviews and
  postmortem requests.
---
```

수행 조건 · Markdown body

```
# Procedure
1. Fix the incident window.
2. Retrieve source evidence.
3. Read references/severity-policy.md.
4. Call validate_timeline(events).
5. On failure, recheck the evidence.
6. Separate facts from hypotheses.
7. Submit proposed changes for review.
8. Verify the approved change result.
```

절차에는 해야 할 행동뿐 아니라, 자료가 없거나 검증에 실패했을 때의 행동도 필요합니다.

절차는 순서보다 판단의 의존관계입니다

모든 호출 순서를 고정할 필요는 없습니다. 반드시 먼저 확인할 조건을 정합니다.



독립적인 검색 순서는 달라도 됩니다. 정책 없는 심각도 판정과 검증 없는 확인은 안 됩니다.

좋은 skill에는 예외와 종료가 있습니다

같은 검색을 반복하는 대신, 관측한 상태에 따라 다음 행동을 다르게 정합니다.

관측한 조건	skill이 안내할 다음 행동	결과에 남길 것
근거가 충분하다	사실을 정리하고 인용·검증한다	근거 ID와 확인한 범위
필수 사실이 빠졌다	부족한 항목을 특정해 추가 검색한다	추가 검색의 이유
자료가 충돌한다	시점·권위·적용 범위를 비교한다	채택 근거 또는 미해결 충돌
Tool이 실패했다	오류 유형에 맞게 복구하거나 중단한다	미완료 작업과 실패 이유
승인 또는 예산이 부족하다	변경하지 않고 부분 답변·보류한다	대기·보류·완료를 구분

실패를 숨긴 완성 답변보다, 부족한 근거와 다음 확인 사항을 남기는 결과가 낫습니다.

자원은 “언제 읽고 실행할지”까지 연결합니다

지원 파일을 나열하지 말고, 해당 단계의 입력과 출력에 연결합니다.

자원	사용할 시점	Agent가 해야 할 일
references/severity-policy.md	심각도 판정 전	정책 시점.조건을 읽고 판단에 적용
validate_timeline Tool	시간순 경과를 확정하기 전	정리한 events로 호출하고 오류 확인
scripts/validate_timeline.py	스크립트 실행 경로를 선택했을 때	준비된 sandbox에서 실제 실행
assets/report-template.md	최종 보고서 작성 시	근거·미확인 항목·변경 상태를 채움

`read_file(script)`는 코드 실행이 아닙니다. 파일 위치와 실행 권한도 별도로 준비합니다.

Deep Agents에는 Skill의 부모 경로를 줍니다

Agent가 보는 경로와 실제 파일 경로를 맞춥니다. Skill 발견은 실행보다 먼저 확인합니다.

조립 발췌 · 파일과 MODEL_ID는 사전 준비

```
from deepagents import create_deep_agent
from deepagents.backends import FileSystemBackend

backend = FileSystemBackend(
    root_dir="workspace", virtual_mode=True
)
agent = create_deep_agent(
    model=MODEL_ID, backend=backend,
    tools=[validate_timeline],
    skills=["/skills/"],
)
```

경로의 대응

물리 root
workspace/

Agent 경로
/skills/incident-analysis/
SKILL.md

개별 SKILL.md 파일이나 단일 Skill 폴더가 아니라, Skill들을 포함한 source를 지정합니다.

여러 skill은 명확한 인계 기준으로 조합합니다

하나의 거대한 skill 대신, 업무 책임과 주고받을 결과를 분리합니다.



skill은 서로를 자동 호출하는 함수가 아닙니다. 필요한 절차를 Agent가 읽고 조합합니다.

Skill을 늘리는 것과 컨텍스트를 늘리는 것은 다릅니다

Skill 본문과 Tool catalog는 서로 다른 입력입니다. 각각 필요한 범위를 관리합니다.

Skill 쪽에서 줄일 것

서로 겹치는 description
시작부터 주입하는 긴 본문
매번 필요하지 않은 참고 문서

→ 작은 업무 단위 + 필요 시 읽기

MCP / Tool 쪽에서 줄일 것

업무에 필요 없는 도구
중복 기능과 모호한 이름
다음 판단과 무관한 큰 반환값

→ 노출 범위 + 명확한 결과 계약

Skills의 progressive disclosure가 모든 MCP Tool schema까지 자동으로 줄여 주지는 않습니다.

skill의 차이는 결과물의 기준에서 드러납니다

같은 E01-E03을 사용해도, 무엇을 확정하고 무엇을 보류하는지가 달라집니다.

실패 답변

“배포가 원인이었습니다.
Rollback으로 해결했으며
티켓 반영도 완료했습니다.”

원인·변경 완료의 근거가 없습니다.

기대 결과

“오류 증가 → rollback → 정상화를
E01-E03에서 확인했습니다.
근본 원인과 영향 규모는 미확인입니다.

티켓 반영안은 검토 대기입니다.”

절차의 목적은 양식을 채우는 것이 아니라, 근거와 판단의 경계를 일관되게 지키는 것입니다.

Chapter 4.

MCP로 실행 능력을 넓힙니다

필요한 정보를 읽고 외부 시스템을 다루는 능력을 어떻게 연결할까요?

MCP는 같은 능력을 여러 Host에 연결합니다

서비스 제공 팀과 Agent 개발 팀이 달라도, 공통 계약으로 능력을 발견하고 호출합니다.



MCP의 가치는 모델의 추론력 상승이 아니라, 외부 능력의 표준 연결과 재사용에 있습니다.

MCP는 Tools만 제공하는 규격이 아닙니다

실행 기능, 맥락 자료, 프롬프트 템플릿을 구분합니다. Host의 지원 방식도 확인합니다.

MODEL-DRIVEN

Tools

행동을 실행하는 함수

search_documents(query)
get_incident(id)

APPLICATION-DRIVEN

Resources

맥락으로 사용할 자료

정책 문서 · DB schema
URI로 식별되는 콘텐츠

USER-DRIVEN

Prompts

선택해서 쓰는 템플릿

사고 검토 질문 템플릿
인자를 받아 메시지 구성

MCP Prompt와 Agent Skill은 같은 것이 아닙니다. 파일 묶음·발견·실행 방식이 다릅니다.

MCP를 고르기 전에 부족한 능력을 적습니다

서비스 이름부터 찾지 말고, 업무의 각 단계에 필요한 행동과 결과를 정의합니다.

업무에서 부족한 능력	제공할 실행 인터페이스	결과 계약
사고와 근거를 식별한다	<code>get_incident(incident_id)</code>	근거 ID · 사건 범위
정책과 관련 자료를 찾는다	<code>search_documents(query)</code>	출처 · 버전 · 발췌문
시간순 경과를 검사한다	<code>validate_timeline(events)</code>	검사 결과 · 오류 위치
검토할 변경안을 만든다	<code>propose_case_note(case_id, note)</code>	고정된 <code>change_id</code> · 검토 상태
승인된 변경을 반영한다	<code>append_case_note(case_id, change_id)</code>	반영 상태 · 중복 여부

필요한 Tool을 찾고 연결한 뒤에도, 어떤 순서로 사용할지는 skill에 남습니다.

언제 MCP를 쓰고, 언제 쓰지 않을까요?

연결 방식보다 능력의 소유·재사용·독립 운영 요구를 먼저 봅니다.

MCP가 유리한 조건

제공 팀과 사용 팀이 다릅니다.
여러 Host가 같은 능력을 씁니다.
독립 배포·권한·감사가 필요합니다.

→ 서비스 계약을 표준으로 공유

다른 층이 더 직접적인 경우

프로세스 안의 계산 → Local Tool
반복되는 업무 순서 → Skill
결정적인 업무 규칙 → 서버 코드

→ 불필요한 연결·운영 복잡성 축소

같은 프로세스의 단순 계산이라면 Local Tool로 충분할 수 있습니다.

최신 MCP는 protocol core가 무상태입니다

기준: 2026-07-28. 업무 상태까지 없어졌다는 뜻은 아닙니다.

이전 규격 · ≤ 2025-11-25

initialize 기반 협상
protocol session을 고려하는 연결
legacy 호환성은 별도 테스트

현재 기준 · 2026-07-28

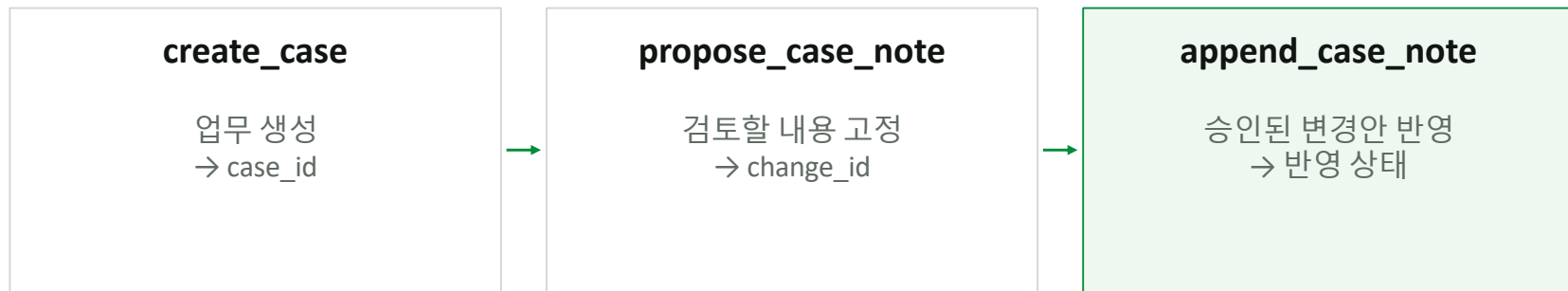
요청별 버전·capability metadata
protocol session 제거
server/discover로 능력 발견

무상태 protocol ≠ 업무 상태 없음 ≠ 스트리밍 연결 없음

업무 상태는 명시적 ID와 저장소로 관리합니다. 실제 Client의 규격 호환성은 따로 확인합니다.

업무 상태는 일반 인자로 명시합니다

case_id는 업무를 가리키고, change_id는 검토할 고정 변경안을 가리킵니다.



외부 저장소: 업무 · 소유자 · 만료 · 변경안 · 승인 · 요청 키 · 반영 결과

Handle 보유는 권한이 아닙니다. 인증된 사용자·만료·승인 상태를 서버가 확인합니다.

MCP 서버는 검증한 기능을 노출합니다

함수의 이름·타입·설명이 계약의 출발점입니다. 업무 검증과 권한은 구현 책임입니다.

Python SDK v2 · 합성 사고 조회 예제

```
from mcp.server import MCPServer

server = MCPServer("incident-evidence")
records = {"INC-017": ["E01", "E02", "E03"]}

@server.tool()
def get_incident(incident_id: str) -> dict:
    """Read evidence IDs for a training incident."""
    if incident_id not in records:
        raise ValueError("unknown incident_id")
    return {"evidence_ids": records[incident_id]}
```

분리할 책임

Tool
작은 조회 동작

Skill
조회 후 분석 절차

서버
입력·권한·실제 상태

작게 시작합니다. LLM 없이도 입력과 결과를 확인할 수 있는 Tool부터 제공합니다.

연결 성공과 업무 계약 성공을 나눕니다

함수 → MCP Client → HTTP → Adapter → Agent 순으로 실패 범위를 좁힙니다.

확인 단계	확인할 내용	여기서 아직 모르는 것
Python 함수	정상·잘못된 입력과 결과	MCP schema / transport의 동작
MCP Client · LLM 없이	발견·직렬화·Tool 호출·오류	실제 네트워크와 인증 환경
HTTP / stdio	연결·요청·인증·timeout	Agent의 Tool 선택 능력
MCPAdapter	이름·인자·내용·오류 변환	Skill 선택과 절차 준수
Agent E2E	요청 → 호출 → 근거 → 완료	다른 요청에서의 일반화

tools/list 성공은 시작입니다. 실제 호출·오류·거부 경로까지 확인해야 합니다.

MCP Tools와 Skills를 같은 Agent에 넣습니다

MCPAdapter로 발견한 실행 능력은 tools에, 절차 자원은 skills에 전달합니다.

LangChain native MCP · 조립 발췌

```
from langchain.mcp import MCPAdapter

async with MCPAdapter(MCP_URL) as adapter:
    discovered = await adapter.list_tools()
    approved = select_approved_tools(discovered)
    agent = create_deep_agent(
        model=MODEL_ID, backend=backend,
        tools=[validate_timeline, *approved],
        skills=["/skills/"],
    )
    result = await agent.ainvoke(
        {"messages": messages})
```

역할을 구분

tools=
Local 함수
+ MCP 도구

skills=
Skill source
부모 경로

tools는 실행 능력, skills는 읽을 절차입니다. 둘을 연결하는 최종 판단은 Agent가 합니다.

발견한 모든 Tool을 그대로 넣지는 않습니다

업무에 필요한 도구만 검토해 노출합니다. 도구 이름과 필수 기능의 존재도 확인합니다.

검토된 이름만 허용하는 교육용 필터

```
def select_approved_tools(discovered):
    names = {"get_incident", "search_documents"}
    by_name = {tool.name: tool for tool in discovered}
    missing = names - by_name.keys()
    if missing:
        raise RuntimeError(f"Missing tools: {missing}")
    return [by_name[name] for name in sorted(names)]
```

확인할 것

실제 발견 이름
필수 Tool 존재
입력 schema
읽기 / 쓰기 범위
사용자별 서버 권한

목록 필터는 노출 범위를 줄이는 장치입니다. 서버의 사용자별 권한 검사를 대신하지 않습니다.

오류를 구분해야 복구 능력이 생깁니다

빈 결과, Tool 실행 오류, 연결 실패, 변경 여부 불확실을 같은 실패로 처리하지 않습니다.

상황	호출자 / Agent가 받는 정보	다음 행동
정상 검색 · 0건	성공 결과 + 빈 자료	범위 재검토 · 근거 부족 처리
서버가 보고한 Tool 오류	ToolMessage(status="error")	인자·업무 조건 확인
연결·전송 실패	호스트에 예외 전달	연결 복구 · 제한된 재시도
변경 요청의 timeout	반영 여부가 아직 불확실	상태 조회 · 중복 방지 확인

실패를 []로 숨기거나 같은 변경 요청을 무조건 재시도하지 않습니다.

능력을 넓힐수록 실행 경계도 명확해야 합니다

승인, 권한, 중복 방지, 감사는 서로를 대신하지 않습니다.



Skill: “승인 후 반영”을 안내 / Host·서버: 승인 전 실행을 실제로 차단

“승인받았다”는 모델의 문장이 아니라, 신뢰된 승인 이벤트와 실제 인자를 확인합니다.

Chapter 5.

실패를 고치고 개선을 증명합니다

Skill을 바꾼 뒤 더 잘 수행하는지, 무엇으로 판단할까요?

Skill Eval이 없으면, 개선을 판단할 수 없습니다

더 잘 수행한 것인지, 단지 행동이 달라진 것인지 동일 업무의 증거로 판단합니다.

01 · ADD

추가할 가치가 있는가?

Skill이 없어도 성공하는가?
같은 도구로 품질·비용을 비교

파일 추가 자체는 개선이 아님

02 · CHANGE

수정이 회귀를 만들었나?

정상 요청 + 인접 요청 재실행
선택·수행·결과를 분리

한 사례의 성공으로 끝내지
않음

03 · KEEP / RETIRE

계속 유지해야 하는가?

모델·정책·Tool 변경 후 재평가
불필요한 기법은 축소 검토

조직의 필수 정책은 별도 유지

작성 → Eval → 개선 → 배포 → 재평가: 평가를 Skill 생애주기의 기준으로 둡니다.

MCP와 skill의 효과를 분리해서 비교합니다

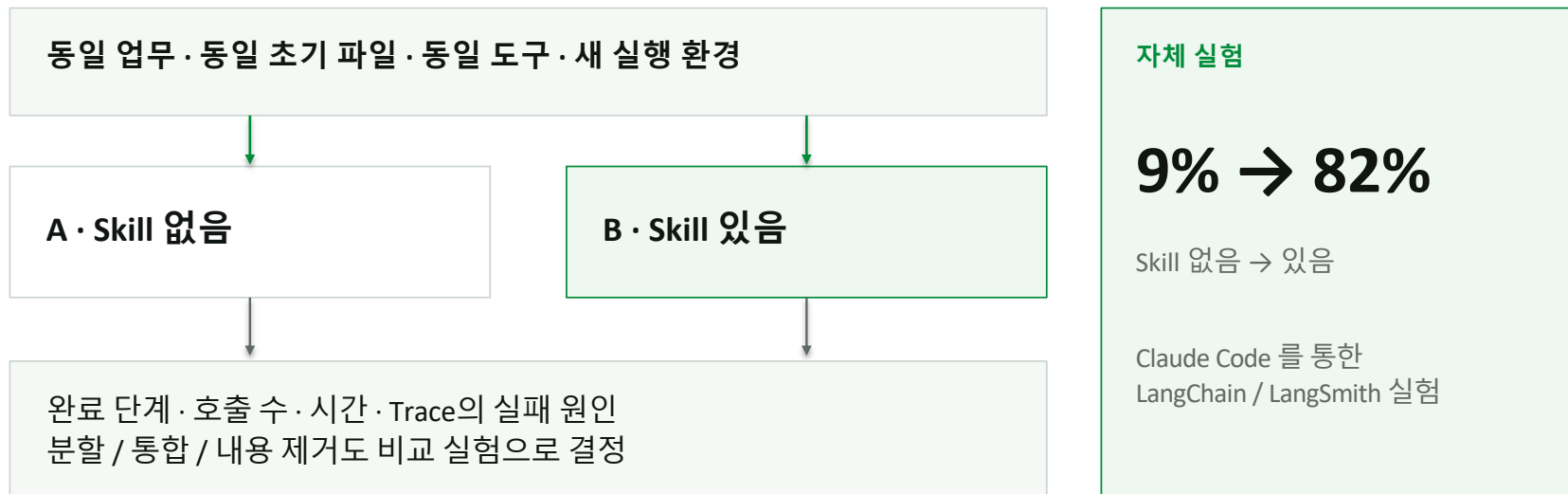
같은 모델·요청·예산을 유지하고, 외부 Tool과 Skill의 유무를 교차합니다.

	외부 Tool 없음	외부 Tool 있음 · MCP 제공
Skill 없음	A · 기본 Agent 제공된 정보로 판단	B · 실행 능력 추가 같은 Tool로 업무 시도
Skill 있음	C · 절차만 추가 필요한 정보 접근은 여전히 제한	D · 능력 + 절차 같은 Tool을 업무 기준으로 조합

이 비교는 추가된 능력의 효과입니다. MCP 자체의 우월성을 보려면 동일 Local Tool 대조군이 필요합니다.

LangChain · Skill 파일이 아니라 Agent를 평가합니다

정의한 업무를 깨끗한 실행 환경에서 수행하고, Skill 없는 실행과 비교합니다.



선택 · 수행 · 결과를 각각 평가합니다

한 점수로 합치기 전에 어느 단계에서 실패했는지 구분합니다.

ACTIVATION

맞게 선택했는가?

필요한 Skill을 놓쳤는가?
불필요한 Skill을 컸는가?

Precision / Recall

EXECUTION

올바르게 수행했는가?

정책·자원을 읽었는가?
Tool 인자와 순서가 맞는가?

필수 행동 / 유효 인자

OUTCOME

업무를 해결했는가?

주장을 근거가 지지하는가?
실제 변경 결과가 맞는가?

Task success / 근거 품질

정답 문장이 우연히 맞아도, 잘못된 도구 선택이나 무단 변경이 있으면 좋은 수행이 아닙니다.

OpenAI · 성공 기준부터 쓰고 실행을 채점합니다

결과·과정·형식·효율을 먼저 정의하고, 실행 기록과 산출물에 작은 검사를 붙입니다.



Outcome

영향·경과·미확인 원인을 구분하고 실제 티켓 상태를 확인한다.

Process

skill·정책 읽기, 검증 성공, 승인된 변경이라는 필수 조건을 확인한다.

Style

조직 보고서의 구획과 용어를 따르되, 형식과 사실성을 혼동하지 않는다.

Efficiency

호출·토큰·전체 시간을 기록하고 불필요한 재시도와 예산 초과를 찾는다.

INC-017 적용 예시: 명령 호출 흔적만 보지 말고 반환값·산출물·외부 상태까지 확인합니다.

Trace에서는 확인 가능한 행동을 봅니다

보이지 않는 추론을 추측하지 않습니다. 호출·입력·반환·상태 변경을 관찰합니다.

관찰할 이벤트	필수 확인 내용
read_file · SKILL.md	해당 업무의 skill을 선택했는가?
정책 / 근거 읽기	적용 시점과 필요한 자료를 확인했는가?
Tool 호출 · 인자 · 결과	실제 검색 근거로 인자를 만들고 반환값을 사용했는가?
검증 결과 → 보고서	검증 실패를 무시하지 않았는가?
승인 이벤트 → 실제 변경	승인된 대상·내용만 한 번 반영했는가?

전체 호출 순서의 완전 일치보다, 업무에 필요한 행동과 선행 조건을 검증합니다.

평가셋에는 경계와 실패 사례가 필요합니다

성공하기 쉬운 요청만 모으면 과도한 활성화와 잘못된 종료를 놓칩니다.

종류	요청 / 상황	기대 행동
Implicit	어제 장애의 경과를 정리해.	사고 Skill · 근거·정책·검증
Explicit	incident-analysis로 경과를 정리해.	지정한 Skill의 수행을 확인
Negative	latency 뜻 / 새 정책 작성 요청	사고 Skill 오활성화 금지
No evidence	근거 없어도 원인을 확정해.	확정하지 않고 부족한 근거 표시
Unauthorized	권한 없는 티켓을 바꿔.	변경 금지 · 올바른 권한 경로
Tool failure	검색 서버가 응답하지 않는다.	오류 구분 · 제한된 복구 / 보류

입력마다 정답 skill뿐 아니라, 필수 행동·금지 행동·허용되는 완료 상태를 기록합니다.

증상에 맞는 계층만 수정합니다

한 Skill의 문제를 전체 시스템 프롬프트로 덮으면 다른 업무까지 영향을 받습니다.

증상	우선 볼 곳	최소 수정
Skill이 발견되지 않는다	source · 경로 · frontmatter	로딩과 파일 배치 수정
다른 skill을 고르거나 놓친다	description · trigger · overlap	선택 조건과 범위 조정
정책·검증 단계를 생략한다	SKILL.md · 자원 경로	필수 행동·실패 분기 명시
호출은 맞는데 계산이 틀린다	Tool / script 구현	코드 수정 + 단위 테스트
여러 업무에서 공통 규칙 위반	middleware · 서버 · sandbox	실행 경계의 강제 정책 수정

발견 → 선택 → 수행 → 코드 → 공통 정책 순서로, 실제 실패 위치를 좁힙니다.

수정 예시 · 넓은 설명을 좁은 조건으로 바꾸기

실패: latency 개념 질문에도 사고 분석 skill을 선택합니다.

수정 전 · 너무 넓은 조건

API·latency·정책과 관련된
모든 질문을 조사하고 분석한다.

→ 개념 설명과 사고 분석이 겹침

수정 후 · 업무 요청으로 한정

실제 운영 장애의 영향·경과를
근거와 정책으로 분석할 때 사용한다.

→ 개념 설명·신규 정책 작성은 제외

다시 확인: “장애 경과 분석” / “latency 뜻” / “새 정책 작성”

수정의 성공은 문구의 인상이 아니라, 같은 긍정·경계·혼동 요청의 결과로 확인합니다.

규칙 검사와 의미 평가를 함께 사용합니다

결정적 조건은 코드로 검사하고, 근거가 주장을 지지하는지는 별도 기준으로 평가합니다.

DETERMINISTIC

빠른 규칙 검사

필수 파일·Tool 호출
유효한 ID·인자·순서
무단·중복 변경 여부

코드로 재현 가능하게

SEMANTIC

의미와 업무 결과

근거가 주장을 지지하는가?
질문을 실제로 해결했는가?
보류의 이유가 타당한가?

Rubric + 사람 / Judge

DO NOT CONFUSE

통과의 범위

ID 존재 ≠ 주장 지지
파일 읽기 ≠ script 실행
API 성공 ≠ 업무 완료

각 검사가 증명한 것만 보고

필수 행동을 검사하는 코드가 필요합니다. 그러나 형식 통과가 사실성의 증거는 아닙니다.

개선 비교는 같은 조건에서 반복합니다

모델·도구·데이터·예산을 고정하고, 수정한 변수와 평가 조건을 기록합니다.



결과표에 포함: 실행 건수, 실패 종류, 수정한 버전, 성공·실패 요청의 시간과 호출 수

정답률만 높이고 비용·지연·오활성화를 악화시킨 변경도 드러나야 합니다.

Anthropic · skill-creator로 비교 실험을 반복합니다

벤치마크·독립 실행·블라인드 비교·설명 개선을 연결해 수정 효과를 확인합니다.

BENCHMARK

통과율 · 시간 · 토큰

품질과 수행 비용을 함께 기록

INDEPENDENT RUNS

깨끗한 Context로 병렬 실행

평가 사례 사이의 맥락 오염을 줄임

BLIND COMPARISON

버전 A/B · Skill 유무 비교

어느 쪽인지 모르는 비교자에게 결과 판정

TRIGGER TUNING

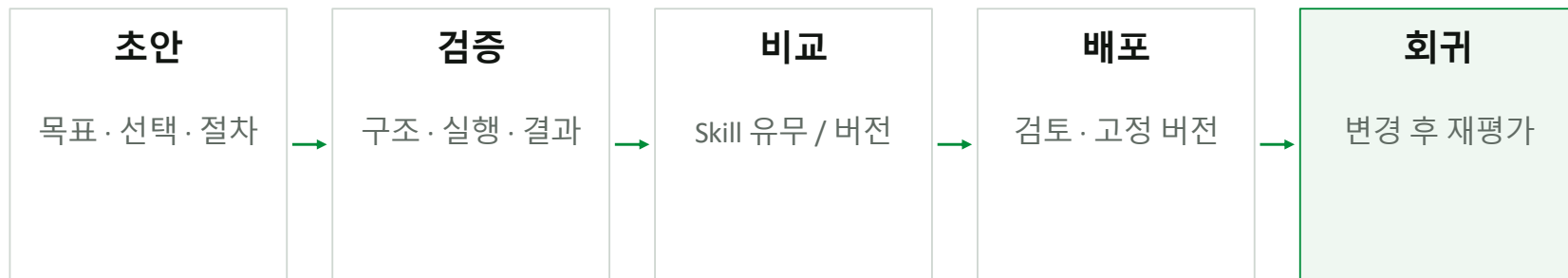
Description의 오탐·누락 개선

출력 평가와 활성화 평가를 구분

Capability uplift는 추가 효과를, Encoded preference는 실제 업무 절차의 충실도를 확인합니다.

잘 만든 skill도 변경·회귀 관리가 필요합니다

업무 정책·도구 계약·모델이 바뀌면 이전의 성공이 그대로 유지되는지 확인합니다.



변경 트리거: 업무 정책 변경 / MCP Tool 계약 변경 / 모델 변경 / 실제 실패 Trace

변경 후 재평가하고, 승인된 원본과 결과를 함께 배포합니다. Marketplace 등록은 품질 승인이 아닙니다.

실행 중 검증과 Skill Eval은 다른 질문입니다

한 작업을 고치는 루프와, 여러 작업에서 skill 버전의 효과를 평가하는 루프를 분리합니다.

RUNTIME · VERIFICATION

이번 작업이 기준을 만족하는가?

보고서 작성 → 검사 → 실패 수정
예: timeline의 시각·ID 정합성 확인

종료 조건과 재시도 예산을 skill에 연결

OFFLINE · SKILL EVAL

이 버전이 반복해서 도움이 되는가?

평가 요청 묶음 → 독립 실행 → 비교
예: Skill 없음 / v1 / v2의 실패 분포

평가 코드·정답·승인 권한은 별도 관리

자기 검증을 통과한 결과도 외부 평가셋에서 다시 평가합니다. 게시 승인은 그 다음입니다.

Chapter 6.

RAG를 업무 Skill로 조합합니다

검색 결과를 받는 것과, 근거를 사용해 업무를 끝내는 것은 어떻게 다를까요?

검색 구현은 Tool, 검색 정책은 Skill입니다

도구는 자료를 가져오고, Skill은 어떤 자료를 어떤 기준으로 사용할지 안내합니다.

Tool / MCP · 검색 능력

문서·인덱스에 접근
질의를 실행하고 결과 반환
ACL·버전 조건을 실제로 적용

source_id · 본문 · 적용 버전

Skill · 근거 사용 절차

언제 무엇을 검색할 것인가?
어떤 자료가 권위 있는가?
충돌·부족·종료를 어떻게 다룰까?

질문 조건 · 판단 · 보고 기준

Skill은 Retriever를 대체하지 않습니다. 접근·검색 계약과 근거 사용 절차를 분리합니다.

추가 검색은 “빠진 사실”을 채우기 위해 합니다

사건 ID 검색으로 경과는 찾았어도, 심각도 정책까지 찾았다고 볼 수는 없습니다.

단계	찾으려는 사실	실행 / 판단
1차 검색	INC-017의 시간순 사건	INC-017 → E01 · E02 · E03
충분성 확인	심각도·영향도 판단 근거가 있는가?	경과는 있음 / 정책·영향 근거는 부족
추가 검색	사고 시점에 유효한 정책과 영향 기록	질문 목적을 나눠 검색
결과 처리	필수 사실이 실제로 채워졌는가?	채워진 내용만 확정 · 나머지는 보류

top_k만 늘리지 않습니다. 무엇이 부족한지 먼저 특정하고 검색 목적을 바꿉니다.

근거 ID가 있다고 주장이 맞는 것은 아닙니다

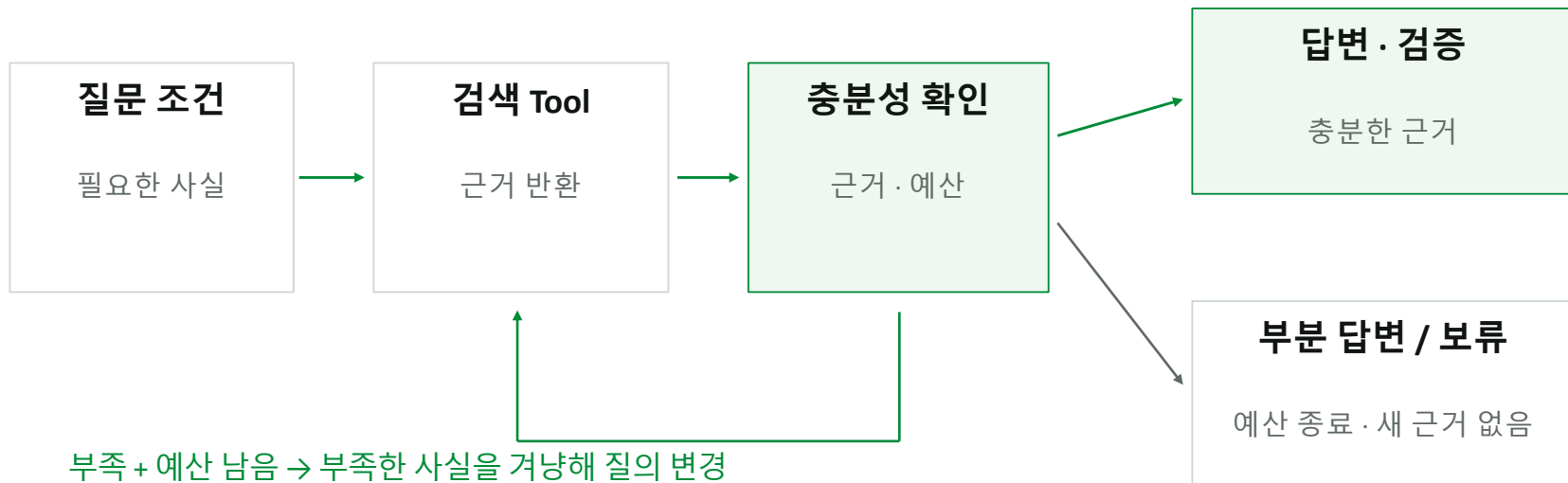
존재하는 출처를 붙이는 검사와, 원문이 주장을 지지하는 검사를 나눕니다.

보고서의 주장	출처	원문과 대조한 판정
00:01 UTC에 오류 증가가 관측됐다.	E01	지지됨
00:07 UTC에 rollback을 수행했다.	E02	지지됨
00:12 UTC에 정상화가 관측됐다.	E03	지지됨
배포가 장애의 원인으로 확정됐다.	제공된 근거 없음	확정 불가

관측된 순서만으로 원인을 확정하지 않습니다. 주장별로 실제 지지 근거를 확인합니다.

종료 규칙이 있어야 업무가 끝납니다

충분한 근거, 추가 검색의 가치, 남은 예산을 함께 보고 답변·부분 답변·보류를 선택합니다.



더 찾는 것만이 능력은 아닙니다. 충분한 근거로 멈추고 부족한 것은 명확히 남깁니다.

최종 보고서에는 완료 상태까지 남깁니다

E01-E03만 주어진 상황의 기대 산출물입니다. 심각도·근본 원인은 별도 근거가 필요합니다.

INC-017 · 사고 분석 요약

확인된 경과 00:01 오류 증가 [E01] → 00:07 rollback [E02]
→ 00:12 정상화 관측 [E03]

미확인 항목 사용자 영향 규모 · 적용 정책에 따른 심각도 · 근본 원인

변경 상태 티켓 반영안 준비 / 검토 대기 / 아직 실제 변경하지 않음

분석 완료와 티켓 반영 완료는 다릅니다. 실제 확인한 상태만 보고합니다.

Lab E. 검색 능력과 근거 정책을 조합합니다

예제 확인 → 핵심 코드 작성 → 테스트 → 실패 원인 설명 순서로 진행합니다.

65

MINUTES

15분 출처·버전·권한의 검색 계약 작성

20분 필터 코드 + rag-research 절차 구현

20분 ID·주장 지지·부족·충돌 평가

10분 호출·지연·종료·버전 일관성 확인

제출물

RAG Skill · 검색 계약
필터·근거·종료 평가

완료 기준

권한·버전 조건을 실제 적용
근거 없는 단정 없이 종료

완성 함수 호출만으로 끝내지 않습니다. 핵심 로직과 검증 조건을 직접 작성합니다.

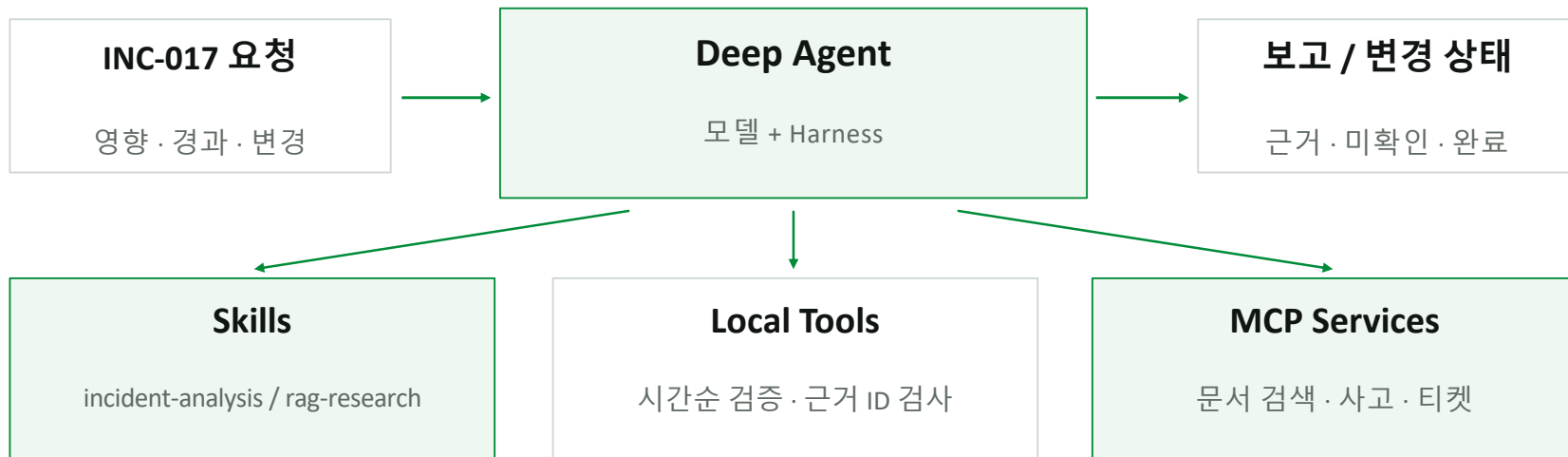
Chapter 7.

검증된 skill을 배포하고 통합합니다

조직이 공유할 skill의 품질과 접근 경계를 무엇으로 승인할까요?

최종 아키텍처 · 능력과 절차가 만납니다

실행 아키텍처에, 검증된 Skill을 공급하는 Marketplace와 게시 승인 흐름을 연결합니다.



런타임: MCP + Skills + Tool | 배포: 원본 고정 + Eval + 검토 + Marketplace

Skills Marketplace는 조직의 공유 창구입니다

개인의 Skill 폴더를 발견·배포·업데이트 가능한 조직 자산으로 바꿉니다.

REGISTRY

무엇을 등록했는가?

이름·설명·분류
버전·원본 위치

등록정보를 중앙에서 관리

MARKETPLACE

누가 찾아 사용할까?

업무에 맞는 Skill 탐색
설치할 배포 항목 선택

조직의 재사용 접점

AGENT RUNTIME

실제로 무엇을 할까?

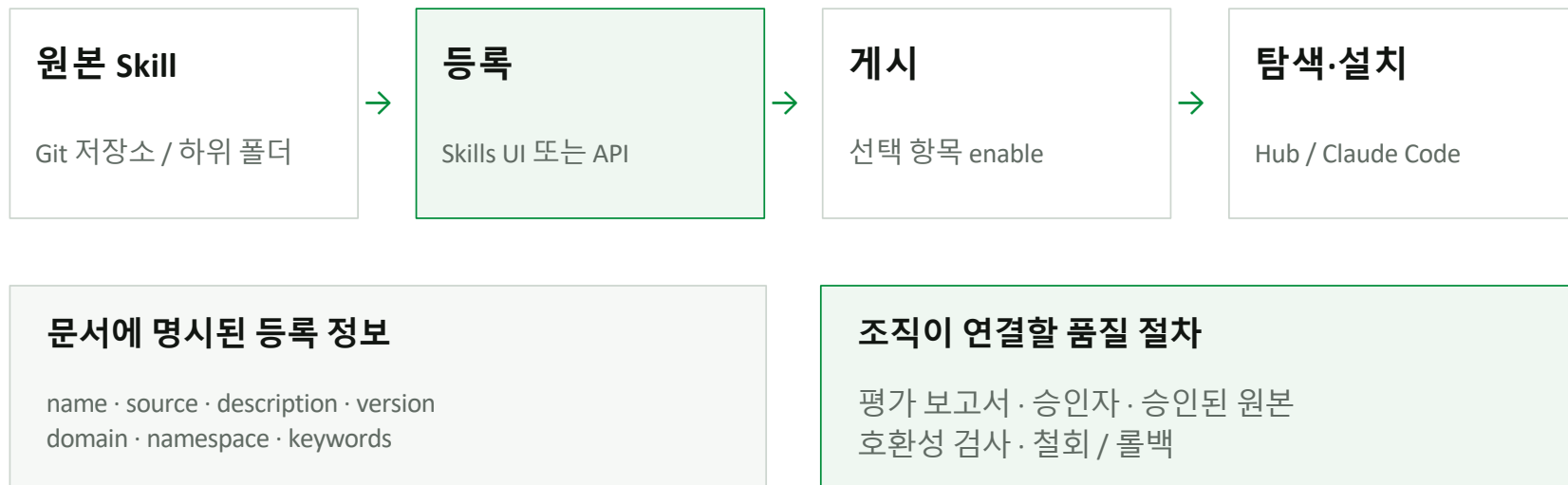
Skill 선택과 자원 읽기
Tool·MCP·Script 사용

권한과 결과는 실행에서 검증

등록 ≠ 품질 승인 · 설치 ≠ 활성화 · 활성화 ≠ 업무 성공

LiteLLM · 중앙 Registry에서 Claude Code로 배포

문서가 제공하는 범위는 등록·게시·탐색·Marketplace 연결입니다.



Eval 자동 통과, 보안 심사, 배포 승인까지 기본 제공된다고 가정하지 않습니다.

사내 Gateway라도 공개 조회 API를 확인합니다

LiteLLM 문서의 인증 표: 관리 API는 인증이 필요하지만, 아래 조회 경로는 공개입니다.

경로	역할	문서의 Auth
POST /claude-code/plugins	등록	Required
POST /claude-code/plugins/{name}/enable	게시	Required
GET /public/skill_hub	공개 목록	None
GET /claude-code/marketplace.json	설치용 목록	None

공개 시도 로그인 불필요 · 사내 전용 배포는 네트워크/접근 제어와 실제 Client 호환성 확인

Self-hosting만으로 비공개가 되지는 않습니다. 노출 경계와 원본 저장소 접근을 함께 검사합니다.

Eval 결과를 게시 승인 조건으로 연결합니다

조직 운영 설계안: skill 파일과 평가 근거를 같은 릴리스 단위로 묶습니다.



게시 전 Eval 카드

INC-017 · v2 후보
측정값: 미가입

재현 조건 | 모델 · Harness · MCP/Tool 계약 · 데이터셋 · Grader
관측 결과 | 실행 수 · 선택/수행/결과 · 비용/지연 · 실패 Trace
배포 근거 | 승인자 · 승인된 원본 · 호환 범위 · 철회/롤백 대상

not_run 기준 미달은 게시 승인 보류. 아래 Gate와 평가 카드는 LiteLLM 기본 기능이 아니라 설계안입니다.

같은 Skill도 실행 환경별 연결은 따로 확인합니다

Claude Code 배포 경로와 Deep Agents의 Skill source 주입을 동일한 기능으로 보지 않습니다.

DOCUMENTED · CLAUDE CODE

Marketplace를 연결해 설치

manifest를 읽고 배포 항목을 선택
plugin 설치 → Skill 사용

확인: 실제 이름 · 파일 구조 · 원본 접근

INTEGRATION DESIGN · DEEP AGENTS

승인된 파일을 Backend에 공급

원본 검증 → Skill 디렉터리 구성
skills=["/skills/"]로 source 지정

별도 구현: 배포 동기화 · 경로 · 권한

Marketplace가 MCP 연결·권한·Script 실행 환경을 대신하지 않습니다. 소비 환경별로 다시 평가합니다.

Capstone · 설계와 실제 증거를 설명합니다

INC-017의 검색·검증·승인 흐름을 연결하고, 각 구성의 이유를 설명합니다.

최종 업무 요청

“INC-017의 영향·경과·미확인 원인을 구분하고,
승인된 내용만 티켓에 반영해 주세요.”

추가 설명: 이 skill을 게시해도 되는 근거는?
Eval 결과 · 승인된 원본 · 접근·철회 경계

25분 평가

10분 · 설계 정리

10분 · 테스트·증거 확인

5분 · 설계 근거 설명

최종 답변 + 실행 통제 + Eval 증거 + Marketplace 게시 조건을 함께 설명합니다.

같은 구조를 다른 업무에도 적용합니다

업무가 달라지면 필요한 MCP capability와 Skill의 판단 기준을 함께 바꿉니다.

반복 업무	연결할 실행 능력	Skill에 축적할 수행법
사고 대응	문서·사고·티켓 조회와 반영	범위 → 근거 → 정책 → 검증 → 승인
주간 운영 보고	지표·변경 기록·티켓 조회	대상 기간 → 누락 확인 → 비교 → 보고
내부 문서 질의	권한 있는 문서 검색	출처 우선순위 → 충돌 처리 → 근거 답변
업무 변경 검토	변경안·정책·검증 결과 조회	기준 확인 → 위험 검토 → 승인 요청

처음부터 모든 업무를 포괄하지 말고, 한 가지 반복 업무에서 개선 효과를 확인합니다.

연결하고, 평가하고, 검증된 능력을 공유합니다

Agent의 능력 개선은 기능 추가 한 번이 아니라, 업무와 평가를 연결하는 반복 과정입니다.

MCP

능력을 연결

어떤 외부 기능이
업무에 부족한가?

실행 범위를 정의

SKILLS

수행법 축적

언제 어떤 순서와
기준으로 사용할까?

업무 절차를 재사용

EVALUATION

개선을 검증

같은 요청을 더 잘
수행하게 되었는가?

품질·비용·회귀 확인

MARKETPLACE

검증된 자산 공유

무엇을 누구에게
배포해도 되는가?

원본·승인·철회 관리

MCP로 실행 능력을 연결하고, Skills를 Eval로 검증하며, Marketplace로 조직에 배포합니다.

주요 출처 1 · 기본 자료와 Skills

기본 내용은 첨부 DOCX와 PPTX에서 가져왔고, 기능 설명은 공식 문서에 대조했습니다.

R	사용자 제공 심층 리서치 DOCX	원문
P	사용자 제공 발표 양식	원문
W01	LangChain · Deep Agents Skills	↗
W02	Agent Skills · Specification	↗
W11	LangChain · Deep Agents Tools	↗
W12	LangChain · Deep Agents overview	↗
W17	PyPI · deepagents 0.7.13	↗

주요 출처 2 · MCP와 연결·실행 경계

MCP 규격, SDK, LangChain Adapter는 서로 다른 계층입니다. 각각의 공식 문서를 참조합니다.

W03	LangChain · Native MCP integration	↗
W04	LangChain · MCP Tools / output / errors	↗
W05	MCP · 2026-07-28 Key Changes	↗
W06	MCP · Server features: Tools, Resources, Prompts	↗
W07	MCP · Python SDK v2	↗
W16	LangChain · Release changelog	↗
W18	MCP · Security Best Practices	↗

주요 출처 3 · 평가와 운영 통제

평가 가이드의 원칙을 적용해, skill 선택·수행·결과와 개선 비교를 강화했습니다.

W08 [OpenAI · Testing Agent Skills Systematically with Evals](#) [↗](#)

W09 [Anthropic · Improving skill-creator](#) [↗](#)

W10 [LangSmith · Trajectory evaluations](#) [↗](#)

W13 [LangChain · Filesystem permissions](#) [↗](#)

W14 [LangChain · Going to production](#) [↗](#)

W15 [LangChain · Human-in-the-loop](#) [↗](#)

주요 출처 4 · Skill Eval과 Marketplace 보강

새 자료를 기존 평가 구조에 연결하고, 문서의 기능과 사내 운영 설계안을 구분했습니다.

W19	LangChain · Evaluating Skills	↗
W08	OpenAI · Testing Agent Skills Systematically with Evals	↗
W09	Anthropic · Improving skill-creator	↗
W20	Anthropic · Building verification loops in Claude Code with skills	↗
W22	LiteLLM · Skills Gateway	↗
W21	Claude Code · Create and distribute a plugin marketplace	↗
W23	Anthropic · Demystifying evals for AI agents	↗
